

CONDITIONALS

LAB 4

SECTION H

SUBMITTED BY:

Simon Samuel

SUBMISSION DATE:

10/3/2025

Problem

The objective of this lab was to write a C program that uses data from the gyroscope and accelerometer of the DualShock 4 controller to establish when the controller is not moving and, if not, on which face it is facing upwards. The output should be in one of six positions: TOP, BOTTOM, LEFT, RIGHT, FRONT, or BACK. We also needed to incorporate two extra features. Then, the program had to end when the triangle button was pressed. Secondly, it should not merely print the orientation over and over again, but only print the orientation when it actually does change. The lab's purpose was mostly just to get more used to conditionals in C and see how numbers from sensors can be translated into information that actually is significant to a person.

Analysis

The inputs to the program were from the controller: the accelerometer's three values (ax, ay, az), the gyroscope's three values (gx, gy, gz), and four button presses (b1, b2, b3, b4). The output was text labels that indicated orientation or alerted the user that the program was finished. To map the inputs onto the outputs, I used the principle that gravity would push one of the axes to +1 or -1 based on the orientation of the controller. The accelerometer value magnitude could be used to infer whether the controller was in motion or not. This mapping wasn't perfect, so I needed to choose tolerance values to allow for minor tilts or wobble. That assumption that the data would never be spotless was essential to the design, with some figuring out I used the tolerance values 0.15 for stillness and 0.25 for orientation.

Design

I divided the problem into little pieces. I started by writing a few functions and building my code around them. I wrote three supporting functions: `mag()` to calculate the length of a three-dimensional vector, `close_to()` to check whether a value was some tolerance away from a point, and `orientation_from_g()` to check which side of the controller was face-up by comparing gx, gy, and gz. Within the loop, the code read in one line of input at a time. I checked whether the controller was still by checking the magnitude of acceleration against zero and some tolerance. When it was still, I employed the orientation function to find the proper side. Then I compared this with the previously printed orientation. If it was a new one, I printed it; else I skipped it so the output will not overwhelm the screen. Finally, I checked the values of the buttons and if triangle was pressed, I broke the loop. At first, this didn't work since I had assumed triangle to be one of the buttons but instead it was another index. I could only identify the right one by experimentally pressing each button separately until I saw which value changed.

Testing

I tested by placing the controller on a table facing upwards with buttons. It printed TOP. Tipping it upside down produced BOTTOM. Putting it on the left or right produced LEFT or RIGHT, and facing forward or backward produced FRONT or BACK. Tipping it at an angle sometimes caused nothing, which meant that the program was not tricked by being in between positions. I also shook the controller, and it didn't print anything since it wasn't stationary. Finally, I pressed triangle, and after I had adjusted the button mapping, the program stopped as it should. I ran these tests a few times to make sure that they worked all the time.

Comments

This experiment was a nice reminder that sensor data is not perfect and requires tolerance values to be effective. Initially my stillness recognition was too strict, so the program virtually never detected the controller to be still. Once I had to adjust the tolerance, it was more realistic. The button mapping also hindered me, because triangle was not where I imagined it was. Once I had realized it, everything else fell into place. Overall, I learned much about how to disassemble small functions to maintain code simplicity and conditionals should be used. Were I to redo anything in the lab, I would have instructions better define button locations to speed up the testing process.

Source code:

```
/*-----  
-           SE/CprE 185 Lab 04 – Conditionals  
- Starter fixed & completed with movement/orientation logic and features.  
-----*/  
  
#include <stdio.h>  
#include <math.h>  
#include <string.h>  
  
/*-----  
-           Prototypes  
-----*/  
  
double mag(double x, double y, double z);  
int  close_to(double tolerance, double point, double value);  
const char* orientation_from_g(double gx, double gy, double gz, double g_tol);  
  
/*-----  
-           Config  
-----*/  
  
static const double MOV_TOL = 0.15;  
static const double G_TOL = 0.25;  
  
#define TRIANGLE_PRESSED(b1) ((b1) != 0)
```

```
/*-----
```

```
-           Implementation
```

```
-----*/
```

```
int main(void) {
```

```
    int t, b1, b2, b3, b4;
```

```
    double ax, ay, az, gx, gy, gz;
```

```
    const char* last_printed = "";
```

```
    while (1) {
```

```
        if (scanf("%d, %lf, %lf, %lf, %lf, %lf, %d, %d, %d, %d",
```

```
                &t, &ax, &ay, &az, &gx, &gy, &gz, &b1, &b2, &b3, &b4) != 11) {
```

```
            break;
```

```
        }
```

```
        double a_mag = mag(ax, ay, az);
```

```
        int is_still = close_to(MOV_TOL, 0.0, a_mag);
```

```
        if (TRIANGLE_PRESSED(b1)) {
```

```
            break;
```

```
        }
```

```
        if (is_still) {
```

```
            const char* ori = orientation_from_g(gx, gy, gz, G_TOL);
```

```

        if (ori && strcmp(ori, last_printed) != 0) {
            printf("%s\n", ori);
            last_printed = ori;
        }
    }
}

return 0;
}

/*-----
-           Helper Functions
-----*/

double mag(double x, double y, double z) {
    return sqrt(x*x + y*y + z*z);
}

int close_to(double tolerance, double point, double value) {
    return fabs(value - point) <= tolerance;
}

const char* orientation_from_g(double gx, double gy, double gz, double g_tol) {
    double ax = fabs(gx), ay = fabs(gy), az = fabs(gz);

    if (ax >= ay && ax >= az) {
        if (close_to(g_tol, 1.0, ax)) {

```

```
        return (gx > 0) ? "RIGHT" : "LEFT";
    }
} else if (ay >= ax && ay >= az) {
    if (close_to(g_tol, 1.0, ay)) {
        return (gy > 0) ? "FRONT" : "BACK";
    }
} else {
    if (close_to(g_tol, 1.0, az)) {
        return (gz > 0) ? "BOTTOM" : "TOP";
    }
}
return NULL;
}
```